



SCARAB: Final Report

For submission to SETU

Author: Stuart Rossiter

Student Number: C00284845

Course: BSc (Hons.) Software Development

Supervisor: Joseph Kehoe

Contents

1. Abstract	4
2. Introduction	5
3. Challenges.....	6
3.1 Breadboard Prototyping Issues.....	6
3.2 Logic Level Translation	6
3.3 NES Mapper Handling.....	6
3.4 GUI Threading with the SCARAB	6
4. Achievements	7
4.1 Arduino Powered Device	7
4.2 GUI Based Cross-Platform Program.....	7
4.3 NES/SNES Support	7
4.4 Save Management	7
4.5 Health Checking.....	7
4.6 Modular Design	8
4.7 EEPROM Identification	8
5. Scrapped Features	9
5.1 Mega Drive Compatibility	9
5.2 Automatic Save Retention Testing	9
5.3 3.3V Compatibility	9
5.4 Identify Corruption	9
6. Lessons Learned	10
6.1 Hardware Prototyping Limitations	10
6.2 “Measure Twice, Cut Once”	10
6.3 Importance of Concurrency in GUI Programs	10
6.4 Benefits of Modular Design	10
6.5 Ethical and Legal Implications of Software.....	10
7. Testing	11
7.1 Precondition Test Cases.....	11
7.1.1 Detect SCARAB	11
7.1.2 Identify Module	12
7.1.3 Identify Cartridge.....	13
7.2 Detailed Test Cases	14
7.2.1 Dump Save.....	14
7.2.2 Restore Save	15
7.2.3 Check Health	16

7.2.4 Identify Corruption	17
7.2.5 Verify Checksum	18
7.2.6 Test Pins.....	19
7.2.7 Test Save Retention	20
7.2.8 Set Pins.....	21
7.2.9 Read Data	21
7.2.10 Write Data	22
8. Reflection	23
9. Glossary	24
10. Bibliography	25

1. Abstract

This final report covers the development cycle of the Save and Cartridge Aid Requiring Adapter Boards. Shortened to SCARAB, the project's purpose is to aid in the preservation of retro gaming hardware, through various health checks.

Throughout this report, the challenges encountered during development will be discussed, along with the methods used to overcome them. These issues range from hardware to software, with an emphasis on the latter as this is a Software Development course.

Following the challenges, the requirements that have been achieved and those that have not been achieved will be discussed. Achievements range from the creation of an Arduino powered device to NES and SNES compatibility, while the scrapped features include the likes of Mega Drive compatibility.

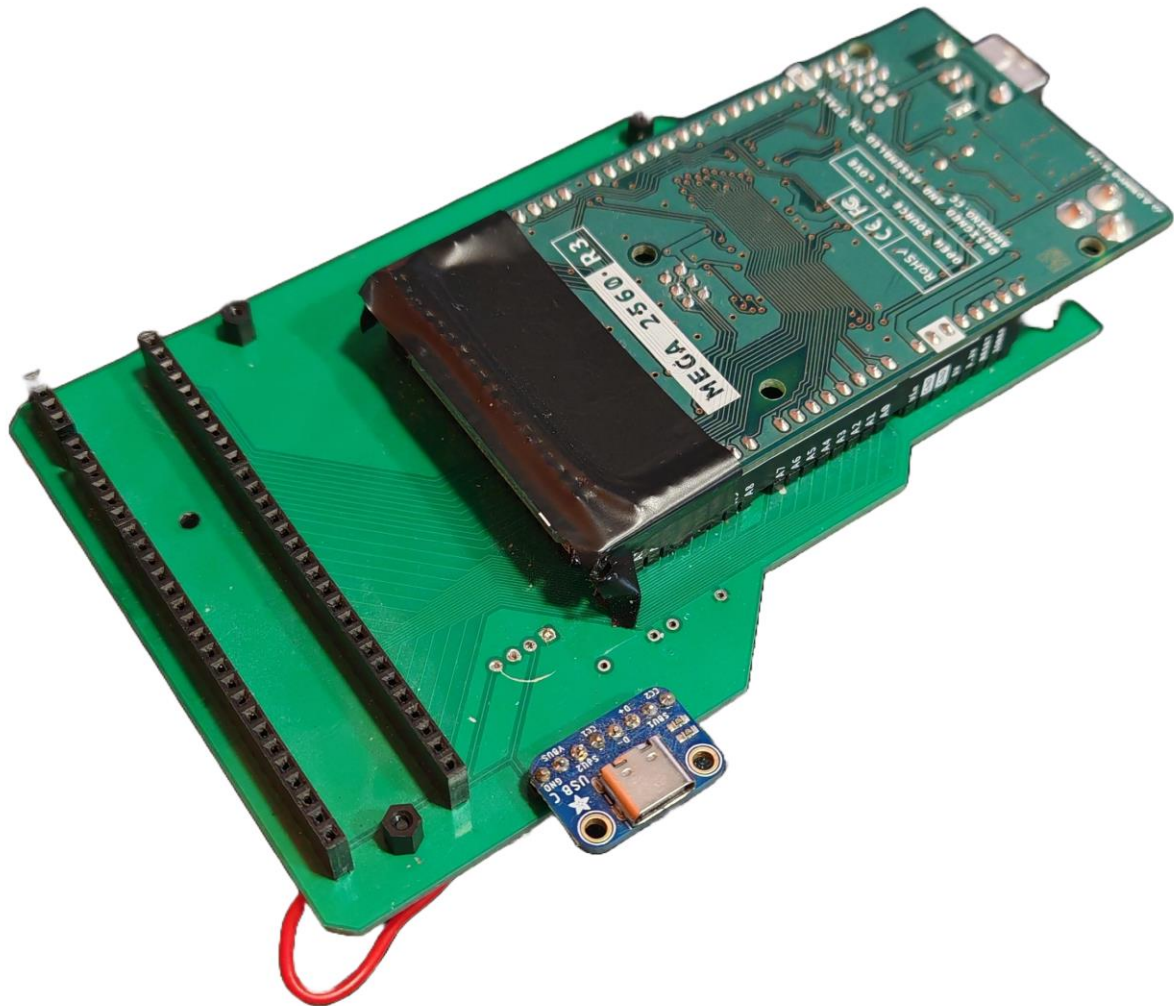
In addition, the lessons learned during the creation of the SCARAB will be discussed. These tie in with the challenges, as many of the challenges provided opportunities to grow and learn as a developer.

Testing is documented next, outlining the rigorous testing which was done on the SCARAB Device and SCARAB PC Program. Many of the test cases cover alternative outcomes, such as a cartridge not having been inserted into the SCARAB.

Finally, a reflection concludes the report. This is a personal section, where my skills and growth as a developer are discussed at length. This final section shows how I have become a better developer after undertaking the SCARAB project.

2. Introduction

The Save and Cartridge Aid Requiring Adapter Boards (SCARAB) is a retro game cartridge health checker, designed to aid in the preservation of the hardware, over the software. The SCARAB project is split into 2 parts: the SCARAB Device, and the SCARAB PC Program.



(Fig 2.1.1 SCARAB board with inserted Arduino Mega 2560)

The SCARAB Device is responsible for interfacing directly with game cartridges and performing tests to determine their current condition. The results of these tests are returned to the user's machine over serial, to be displayed in the PC Program.

The SCARAB PC Program provides a user interface for the device, communicating with it via serial. Core features include cartridge identification, a health checking suite, and save management capabilities.

3. Challenges

3.1 Breadboard Prototyping Issues

One of the main challenges during the development of the SCARAB, was the prototyping of modules. The rows of pins on the Cartridge Connectors were too close together to utilise a breadboard, without connecting both rows of pins to the same side of said breadboard. In addition to this, the pins were slightly too small to have a snug fit within a wire's female end. This would result in loose connections, causing insufficient power or bad data reads/writes to cartridges. To facilitate the ease of development, only the most fundamental features of the modules would be tested on breadboards, i.e. reading a byte from and writing a byte to a cartridge. With a simple prototype confirming viability of the module, the PCB would be designed and ordered to fully develop the module, and the SCARAB's interface with it.

3.2 Logic Level Translation

While the Arduino is powered at 5v and utilises the same voltage for its logic levels, not all cartridges support this level of power. NES, SNES, and GB/GBC cartridges run natively on 5v, yet the N64 and GBA cartridges run at a lower voltage: 3.3v. Translating all GPIO pins from 5v down to 3.3v would be necessary for these cartridges, which proved to be more difficult than anticipated. Attempts were made to utilise the SN74LVC245AN Octal Bus Transceivers, as it was believed that they would suffice. This was not the case, as the level shifting capabilities of these were not bi-directional. The voltage would be downgraded to 3.3v coming from the Arduino, but it would not be upscaled back to 5v when coming from the cartridge. Alternative approaches were considered, to ensure safe voltage translation and communication with low voltage cartridges.

3.3 NES Mapper Handling

NES cartridges use a wide variety of mapper configurations. These mappers are specialized hardware chips within the cartridges that extend the console's capabilities. For example, they can swap in different banks of ROM/RAM data, allowing games larger than 40KB to be run on the NES. These mappers all behave differently, and as such, performing operations on the cartridges requires pre-operation mapper handling. The decision was made to stick with 6 main mappers, for the time being: NROM, UNROM, CNROM, AOROM, MMC1, and MMC3. Of the 695 licensed releases in North America, 668 are covered by these mappers (tepples, 2020), providing broad compatibility while keeping the complexity to a minimum.

3.4 GUI Threading with the SCARAB

When interfacing with the SCARAB device, I encountered the issue of the PC Program's main GUI Thread becoming blocked, while the SCARAB performed its tests on the cartridge that was inserted. This was a major problem, as the app would become completely unresponsive while the tests were being run. To remedy this, different approaches to concurrency were investigated, including traditional multi-threading and asynchronous execution. In the end, a solution was devised using PySide6's QTimer.singleShot method. This method utilises a delayed callback to a function, without blocking the main thread. To implement, the operations were split into discrete stages, progressing to the next stage via these scheduled executions.

4. Achievements

4.1 Arduino Powered Device

One of the 2 main components of the SCARAB project is the SCARAB device. A fully functional cartridge interfacing device was designed and implemented, powered by an Arduino Mega 2560. The main PCB also hosts a USB-C module for power supply, a buck converter for converting the 5V supply to 3.3V for certain modules, and pin headers for attaching module boards. The device reads from cartridges based on the inserted board, and relays the necessary information to the SCARAB PC Program.

4.2 GUI Based Cross-Platform Program

The other component of the SCARAB project is the SCARAB PC Program: a GUI-based application designed and developed in Python using the PySide 6 framework. The application supports user-provided cartridge imagery, displayed when the specified cartridge is inserted. Key details about the cartridge are also displayed to the user, such as Title, ROM Size, and Checksum. The SCARAB application has also been developed with modularity in mind. It allows for new modules to be added by placing their Python files in a designated Modules directory, which allows for extension without modifying the application.

4.3 NES/SNES Support

The SCARAB module boards allow for future expansion, and the support of various cartridge types. Currently, the SCARAB supports 2 cartridge types: the NES, and the SNES. These modules were designed and prototyped on breadboards, before being given custom PCBs to connect to the SCARAB device. The NES has support for 6 mappers, allowing for a high level of compatibility with licensed releases. The SNES supports LoRom and HiRom layouts, covering a vast majority of cartridges, save for those with the SA-1 enhancement chip.

4.4 Save Management

As part of the cartridge aiding functionalities of the SCARAB, the ability to dump/restore save data from cartridges was included to, for example, allow the batteries powering internal SRAM to be changed without losing one's save file. This save management system was developed and integrated into the SCARAB PC Program, providing means to backup and archive save files for cartridge repair or otherwise.

4.5 Health Checking

The main purpose of the SCARAB is to allow for users to check the overall health of their retro video game cartridges. This is facilitated by the SCARAB Program's Check Health Suite. The checking of cartridge health is split into 3 distinct categories; Cartridge Pins, ROM Data, and Save Data.

For Cartridge Pins, the functionality of the cartridge's edge connector's pins are verified. The SCARAB tests that all data pins are toggling on and off as intended, which consequently tests the READ and ROMSELECT pins, if applicable.

For ROM Data, the checksum of the ROM is calculated. This varies by system, as some contain a checksum within the ROM header, whereas some need to be validated externally. For example, the SNES header contains 16-bit sum of every byte of ROM data, with the overflow discarded (Nintendo, 1993).

Finally, the Save Data testing consists of the testing of the cartridge's save retention. Any previously existing save file is dumped from the cartridge, and a predetermined save is written to it. After cutting cartridge power for ~30 seconds, the predetermined save is read back and compared to the original. The match percentage is returned to the user.

4.6 Modular Design

The SCARAB device supports a modular design. The cartridge connectors each have their own custom PCB, which they are attached to along with an EEPROM for identification. This modular system allows for users to pick and choose which cartridges they wish for their SCARAB device to support. In addition, this allows for future expansion, for cartridge types currently not supported by the SCARAB.

4.7 EEPROM Identification

Each SCARAB module accommodates an EEPROM for cartridge identification. The first page of the EEPROM stores an 8-byte identifier string, specific to each module. The SCARAB Program provides the functionality to set this identifier when performing initial setup for newly assembled modules.

5. Scrapped Features

5.1 Mega Drive Compatibility

Originally, the SCARAB was to come with a Mega Drive Adapter Board, which would allow for the analysis and archival of SEGA Mega Drive cartridges. This, however, was dropped due to time constraints. The choice was made to drop the SEGA Mega Drive over other cartridge types, as it is less popular than the Nintendo consoles planned for the SCARAB (Zackariasson & Wilson, 2012).

5.2 Automatic Save Retention Testing

When testing a cartridge's save data retention, the SCARAB requests that the user unplug the power cable, to allow for RAM to lose the data written to it. The original plan was for this to be an automatic process.

Using MOSFETs, the SCARAB would control the power lines. Unfortunately, the SCARAB's main board was very packed, leading the idea to be pushed back to a later stage. This later stage never arrived, leading to the scrapping of the MOSFET controlled power lines. With a redesign, this could potentially be accommodated.

5.3 3.3V Compatibility

The SCARAB was ideally to support cartridges of both 5V logic levels, and 3.3V logic levels. This did not end up happening. The 8-bit Logic Level Translators used, the SN74LVC245A Octal Bus Transceivers, were not ideal for the job. It was my understanding that the level shifting was bidirectional, but that is not the case. The use of a truly bidirectional level shifter, in the TXS1080E would serve to allow for such compatibility. By omitting the 3.3V environment from the SCARAB's compatibility, it immediately renders the Nintendo 64 and Game Boy Advance adapter boards redundant until a redesign is done.

5.4 Identify Corruption

Out of the 4 tests originally planned as part of the Check Health Suite provided by the SCARAB, only 3 were completed. Identify Corruption was scrapped. The original plan was to identify any areas of ROM that were incorrect or had become corrupted. This proved to be far more difficult than thought. Without a fully intact version of the ROM on the inserted cartridge, there would be no way to know exactly which bytes were invalid. Alternatives were explored, such as adding a feature for users to supply their own ROM files. This too was abandoned, as I felt it condoned piracy, which is a moral grey area.

6. Lessons Learned

Throughout the development of the SCARAB, I have learned many things.

6.1 Hardware Prototyping Limitations

Throughout the development of the SCARAB device, it became clear that there were limitations to what could be achieved via breadboard prototyping. Loose wires, cheap breadboards, and multiple connections between components create additional potential points of failure. As a result, I have learned that the ideal approach is to create a prototype of the most basic features on a breadboard, and to develop fully on a designed PCB.

6.2 “Measure Twice, Cut Once”

Upon receiving the PCBs I had designed, it had come to my attention that the spacing between the pins on the GB/GBC/GBA modules were incorrect. The pin distance for these devices is 1.5mm, yet I had mistakenly set them to the pre-existing distance of 1.27mm. With no time to reprint, the module had to be abandoned. This taught me the incredibly important lesson of double and triple checking all my work.

6.3 Importance of Concurrency in GUI Programs

When running the tests on the inserted cartridges, I noticed that the SCARAB PC program became unresponsive. Concerned, I investigated and found that it was a quirk of the main GUI thread of PySide6 being occupied by the testing. I learned many ways to remedy this, such as external multithreading, or Qt Event Loop functions within PySide6. My chosen method was the `QTimer.singleShot` function which allows for delayed callbacks. This was a vital change, as letting a GUI go unresponsive results in a poor user experience.

6.4 Benefits of Modular Design

A key feature of the SCARAB is its modular design. It allows for future expansion, allowing users to create and implement their own SCARAB modules with ease. This also helped me in my development, as it led to a fully functional GUI early on, where I could simply drop in new modules to test.

6.5 Ethical and Legal Implications of Software

When deliberating the feasibility of the Identify Corruption test, the idea of users supplying their own valid ROMs was brought up. This raised an ethical dilemma about whether this was encouraging piracy. This also crosses over into the legality of such an option.

7. Testing

The SCARAB project was tested rigorously, as outlined below. Prior to all these tests, the following preconditions were verified and assumed to be handled correctly throughout all subsequent tests.

7.1 Precondition Test Cases

7.1.1 Detect SCARAB

Actors: PC, Cartridge Port	
Pre-conditions:	The PC is running the SCARAB GUI program.
Main Success Scenario:	1. The user plugs the SCARAB into the PC using the USB cables. 2. The user clicks “Re-Identify SCARAB” 3. The program begins its attempt of identifying the SCARAB.
Expected Result:	The SCARAB PC Program has identified the SCARAB.
Actual Result:	The SCARAB was indeed correctly identified, and reflected in the GUI Program.
Result:	PASS
Comments:	
Alternate Scenario	
Scenario	The SCARAB was not plugged in to the PC.
Expected Result:	The SCARAB PC Program displays an error message to the user.
Actual Result:	The user receives an error message as expected, notifying them that the SCARAB was not found.
Result:	PASS
Alternate Scenario	
Scenario	An Arduino Mega 2560 is connected, but without the SCARAB code on it.
Expected Result:	The SCARAB PC Program displays an error message to the user.
Actual Result:	The user receives an error message as expected, notifying them that the SCARAB was not found.
Result:	PASS

7.1.2 Identify Module

Actors: PC, Cartridge Port	
Pre-conditions:	The PC is running the SCARAB GUI program. The SCARAB has been plugged in and identified.
Main Success Scenario:	1. The user plugs a module into the SCARAB's module connector. 2. The user clicks "Modules" on the sidebar of the GUI program, moving to the "Modules" menu. 3. The user clicks "Identify Module". 4. The SCARAB attempts to identify the inserted module.
Expected Result:	The SCARAB has identified the module.
Actual Result:	The SCARAB module was correctly identified, and this was reflected accordingly in the program.
Result:	PASS
Comments:	
Alternate Scenario	
Scenario	No module has been inserted into the SCARAB.
Expected Result:	The SCARAB PC Program displays an appropriate error message to the user.
Actual Result:	The user received an error message as expected, notifying them that no module has been inserted.
Result:	PASS
Alternate Scenario	
Scenario	The inserted module has not had its EEPROM configured.
Expected Result:	The SCARAB PC Program displays an appropriate error message to the user.
Actual Result:	The user received an error message as expected, notifying them that no module has been inserted.
Result:	PASS

7.1.3 Identify Cartridge

Actors: PC, Cartridge Port	
Pre-conditions:	The SCARAB is plugged into the PC. The inserted module has been identified.
Main Success Scenario:	1. A cartridge is inserted into the SCARAB's module. 2. The user clicks "Modules" on the sidebar of the GUI program, moving to the "Modules" menu. 3. The user clicks "Identify Cartridge". 4. The SCARAB attempts to identify the inserted cartridge.
Expected Result:	The cartridge is successfully identified and reflected in the GUI program.
Actual Result:	The cartridge was successfully identified, with details about it being reflected to the user.
Result:	PASS
Comments:	
Alternate Scenario	
Scenario	No cartridge is inserted.
Expected Result:	The SCARAB PC Program displays an appropriate error message to the user.
Actual Result:	The user received an error message as expected, notifying them that there is no cartridge present.
Result:	PASS

7.2 Detailed Test Cases

7.2.1 Dump Save

Actors: User, SCARAB Device	
Pre-conditions:	The SCARAB is connected to a PC with both USB cables. The PC is running the SCARAB GUI program. A cartridge port module is inserted into the SCARAB. A cartridge is inserted into the SCARAB's cartridge port. A 100% save file has been restored to the cartridge in question.
Main Success Scenario:	1. The user selects the "Save Management" option from the GUI's sidebar 2. The program takes the user to the "Save Management" screen. 3. The user selects "Dump Save Data" from the main window. 4. The program begins dumping the save data of the inserted cartridge.
Expected Result:	The SCARAB has successfully dumped the save file of the inserted cartridge to a new file on the PC.
Actual Result:	A .sav file was successfully created, and the validity was verified against the pre-restored save file.
Result:	PASS
Comments:	
Alternate Scenario	
Scenario	The inserted cartridge does not support save data.
Expected Result:	The user is notified that the cartridge does not support save data, and the program does not attempt to dump the save.
Actual Result:	The user received an error message as expected, notifying them that save data is not supported.
Result:	PASS

7.2.2 Restore Save

Actors: User, SCARAB Device	
Pre-conditions:	The SCARAB is connected to a PC with both USB cables. The PC is running the SCARAB GUI program. A cartridge port module is inserted into the SCARAB. A cartridge is inserted into the SCARAB's cartridge port.
Main Success Scenario:	1. The user selects the "Save Management" option from the GUI's sidebar 2. The program takes the user to the "Save Management" screen. 3. The user selects "Restore Save Data" from the main window. 4. The program prompts the user to select a save file, displaying the dumped saves compatible with the inserted cartridge. 5. The user selects the predetermined "100% completion" save file, and clicks "Restore Save" 6. The SCARAB begins restoring the save file to the cartridge.
Expected Result:	The SCARAB successfully restores the save data to the cartridge.
Actual Result:	The save data was confirmed to have been written, after having been checked in-game.
Result:	PASS
Comments:	
Alternate Scenario	
Scenario	The inserted cartridge does not support save data.
Expected Result:	The user is notified that the cartridge does not support save data, and the program does not attempt to restore the save.
Actual Result:	The user received an error message as expected, notifying them that save data is not supported.
Result:	PASS

7.2.3 Check Health

Actors: User, SCARAB Device	
Pre-conditions:	The SCARAB is connected to a PC with both USB cables. The PC is running the SCARAB GUI program. A cartridge port module is inserted into the SCARAB. A cartridge is inserted into the SCARAB's cartridge port.
Main Success Scenario:	1. The user selects the "Check Health" option from the GUI's sidebar. 2. The program takes the user to the "Health Management" screen. 3. The user clicks "Check Health". 4. The program brings the user to the testing screen, where it warns the user not to remove the cartridge during testing, while providing progress updates. 5. The SCARAB performs the test(s) on the cartridge.
Expected Result:	The tests are run by the SCARAB, and the results are displayed.
Actual Result:	The tests were successfully run by the SCARAB, and the results were displayed within the GUI Program.
Result:	PASS
Comments:	

7.2.4 Identify Corruption

Actors: User, SCARAB Device	
Pre-conditions:	The SCARAB is connected to a PC with both USB cables. The PC is running the SCARAB GUI program. A cartridge port module is inserted into the SCARAB. A cartridge is inserted into the SCARAB's cartridge port.
Main Success Scenario:	1. The user selects the "Check Health" option from the GUI's sidebar. 2. The program takes the user to the "Health Management" screen. 3. The user selects "Custom Scan" 4. The user ticks the box for Identify Corruption. 5. The user clicks "Run Test(s)". 6. The program brings the user to the testing screen, where it warns the user not to remove the cartridge during testing, while providing progress updates. 7. The SCARAB checks the ROM and save data for any corruption.
Expected Result:	The SCARAB displays the results of the Identify Corruption test to the user.
Actual Result:	The Identify Corruption test never ran, as it always defaults to "Not Selected."
Result:	FAIL
Comments:	Feature was scrapped due to difficulties and ethical concerns.

7.2.5 Verify Checksum

Actors: User, SCARAB Device	
Pre-conditions:	The SCARAB is connected to a PC with both USB cables. The PC is running the SCARAB GUI program. A cartridge port module is inserted into the SCARAB. A cartridge is inserted into the SCARAB's cartridge port.
Main Success Scenario:	1. The user selects the "Check Health" option from the GUI's sidebar. 2. The program takes the user to the "Health Management" screen. 3. The user selects "Custom Scan" 4. The user ticks the box for Verify Checksum. 5. The user clicks "Run Test(s)". 6. The program brings the user to the testing screen, where it warns the user not to remove the cartridge during testing, while providing progress updates. 7. The SCARAB calculates the checksum of the cartridges rom, and checks it against a known valid checksum.
Expected Result:	The result of the Verify Checksum test is returned to the user. Known functioning cartridges should pass.
Actual Result:	The SCARAB Program successfully relayed the test results back to the user through the GUI. The known working cartridge passed.
Result:	PASS
Comments:	

7.2.6 Test Pins

Actors: User, SCARAB Device	
Pre-conditions:	The SCARAB is connected to a PC with both USB cables. The PC is running the SCARAB GUI program. A cartridge port module is inserted into the SCARAB. A cartridge is inserted into the SCARAB's cartridge port.
Main Success Scenario:	1. The user selects the "Check Health" option from the GUI's sidebar. 2. The program takes the user to the "Health Management" screen. 3. The user selects "Custom Scan" 4. The user ticks the box for Test Pins. 5. The user clicks "Run Test(s)". 6. The program brings the user to the testing screen, where it warns the user not to remove the cartridge during testing, while providing progress updates. 7. The SCARAB reads tests the toggling of data pins using the header or other known ROM data.
Expected Result:	The results of the test are displayed to the user. Known functioning cartridges should pass.
Actual Result:	The results of the test were relayed back to the user through the GUI. A known working cartridge passed.
Result:	PASS
Comments:	

7.2.7 Test Save Retention

Actors: User, SCARAB Device	
Pre-conditions:	The SCARAB is connected to a PC with both USB cables. The PC is running the SCARAB GUI program. A cartridge port module is inserted into the SCARAB. A cartridge is inserted into the SCARAB's cartridge port.
Main Success Scenario:	1. The user selects the "Check Health" option from the GUI's sidebar. 2. The program takes the user to the "Health Management" screen. 3. The user selects "Custom Scan" 4. The user ticks the box for Test Save Retention. 5. The user clicks "Run Test(s)". 6. The program brings the user to the testing screen, where it warns the user not to remove the cartridge during testing, while providing progress updates. 7. The SCARAB backs up the user's save and writes a pre-determined save file to the cartridge. 8. The user, as prompted, removes the USB-C Power Cable from the SCARAB. 9. After 30 seconds, the user reinserts the power cable, as prompted. 10. The save file is dumped and compared to the pre-determined one. 11. The backup save file is rewritten to the cartridge.
Expected Result:	The results are a >99% match.
Actual Result:	The save files were a >99% match, confirming an operational save battery.
Result:	PASS
Comments:	
Alternate Scenario	
Scenario	User does not unplug the USB-C Power Cable.
Expected Result:	The results were skewed, as a near 100% match.
Actual Result:	The results are a 99.99% match
Result:	PASS

Alternate Scenario	
Scenario	The user does not plug the USB-C Power cable back in.
Expected Result:	The results are skewed, as a near 0% match
Actual Result:	The results were a 0% match.
Result:	PASS
Alternate Scenario	
Scenario	The inserted cartridge does not support save data.
Expected Result:	The user is notified that the cartridge does not support save data, and the test is listed as “Not Selected”.
Actual Result:	The test was listed as “Not Selected”.
Result:	FAIL
Comments:	User is not notified about lack of save data support.

7.2.8 Set Pins

Actors: None	
Pre-conditions:	The SCARAB is powered. The SCARAB has received a request
Main Success Scenario:	1. The SCARAB checks the correct pin layout for the inserted port module. 2. The SCARAB sets the appropriate pins to input or output.
Expected Result:	The correct ports are set to input and output.
Actual Result:	The correct ports were set on the SCARAB.
Result:	PASS

7.2.9 Read Data

Actors: PC, Cartridge Port	
Pre-conditions:	The SCARAB is plugged into the PC. The SCARAB has received a read request from the PC.
Main Success Scenario:	1. The SCARAB receives the address to read from, and the size of the read. 2. The SCARAB reads the data from the cartridge into a buffer.
Expected Result:	The SCARAB reads data from the given address and returns it to the PC over serial.
Actual Result:	The SCARAB read the data and returned it, which was validated against a known valid ROM.
Result:	PASS

7.2.10 Write Data

Actors: PC, Cartridge Port	
Pre-conditions:	The SCARAB is plugged into the PC. The SCARAB has received a write request from the PC.
Main Success Scenario:	1. The SCARAB receives the request data. 2. The SCARAB loads the writable data into a buffer. 3. The SCARAB writes the data to the cartridge, beginning at the specified address.
Expected Result:	The SCARAB successfully writes the necessary data to the cartridge.
Actual Result:	The SCARAB wrote the data to the cartridge, tested by writing to the save data and checking in-game
Result:	PASS

8. Reflection

During my development of the SCARAB, I learned a lot about the development cycle of software, and the intricacies of embedded systems. While some decisions I stand by, there are some things that I would change were I to do this project all over again.

First and foremost, I would swap out the Arduino Mega 2560 Rev 3 for the Arduino Mega PRO. The PRO supports being powered at 3.3v, which negates the need for logic level translators altogether. This is seen in the likes of the Open-Source Cart Reader by Sanni (sanni, 2025).

In addition, I would reduce the number of USBs used from 2 to 1. Currently, the Arduino gets powered by its serial cable, with it also serving as the Serial Communications cable. The cartridges get powered by an external USB-C Breakout board. Ideally, I would develop a communications protocol and utilise the USB-C port for both power and data transfer.

The SCARAB introduced challenges previously unknown to me, such as the ethical concerns of encouraging piracy. This taught me to think about more than just the feasibility of coding a feature, when it came to deliberating whether it should be included.

To conclude, the most meaningful takeaway from the SCARAB project was what it revealed about what I value as a developer. I learned to love coding more than I had ever believed, when working on something I was truly interested in, that being retro gaming and hardware. Through the earlier years of the Software Development course, heavy amounts of web development were involved, which I did not find as interesting as one would hope. The SCARAB, however, has reignited my passion for development, and has outlined for me the type of work I would like to do going forward.

9. Glossary

SCARAB – Save and Cartridge Aid Requiring Adapter Boards

SNES – Super Nintendo Entertainment System

NES – Nintendo Entertainment System

ROM – Read Only Memory

EEPROM – Electronically Erasable Programmable Read Only Memory

USB – Universal Serial Bus

10. Bibliography

Nintendo, 1993. *SNES Development Manual*. s.l.:Nintendo of America.

sanni, 2025. *OSCR: What to Order*. [Online]

Available at: <https://github.com/sanni/cartreader/wiki/What-to-order>

[Accessed 20 April 2026].

tepples, 2020. *Most common mappers*. [Online]

Available at: <https://forums.nesdev.org/viewtopic.php?t=20019>

[Accessed 20 April 2026].

Zackariasson, P. & Wilson, T. L., 2012. *The Video Game Industry: Formation, Present State, and Future*. 1st ed. New York: Routledge.